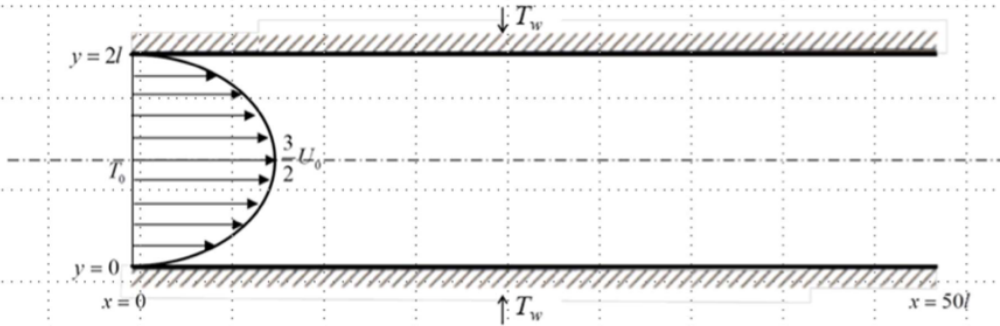


Term Project #1 20244109 기계공학과 권혁준



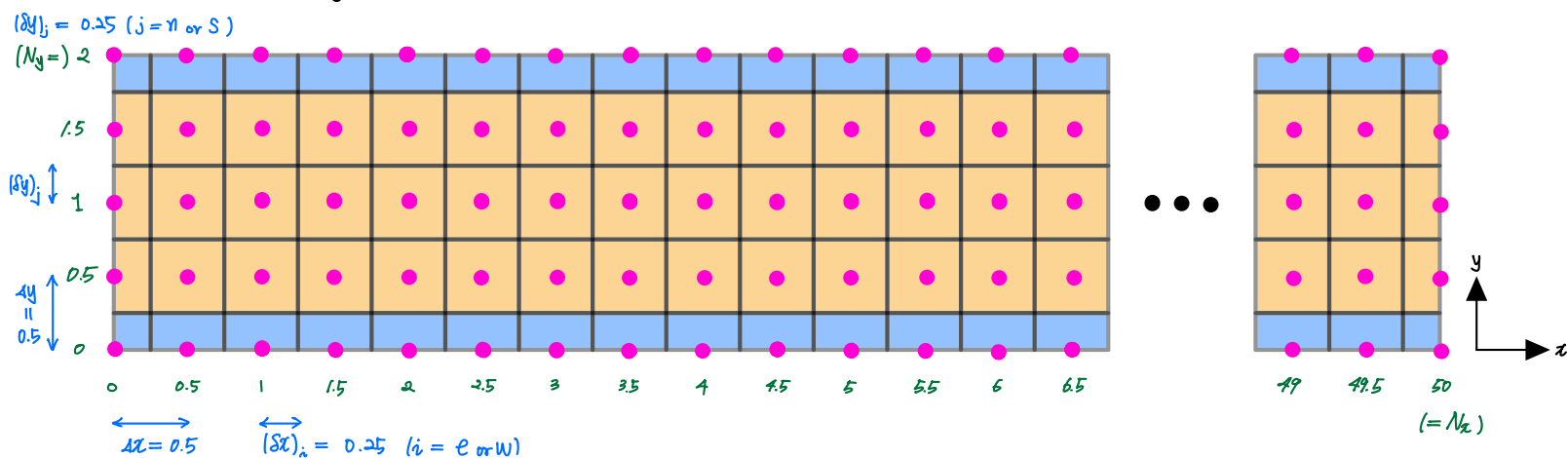
Consider steady laminar flow between two parallel plates spaced $2l$ apart and subject to uniform wall temperature ($T_w = 400$ K). The velocity profile is assumed to be fully-developed. The inlet temperature of the air T_0 is 300 K. We wish to determine the temperature and Nusselt number distributions for moderate values of the Reynolds number and the Peclet number using the control volume method.

- Determine the appropriate boundary conditions.
- Obtain the discretization equations.
- Plot the temperature distributions at the Reynolds number of 50.
- Compare the results for the Nusselt number development with available data.

Let's set $l = 1$. $\rightarrow 2l = 2$, $50l = 50$

I discretized the domain as follows. (Boundary : half-CV, Interior : full-CV)

Ex) When $\Delta x = \Delta y = 0.5$



(a) Boundary conditions

- | | | | |
|---|---|--|---|
| { | bottom wall : Dirichlet Boundary Condition ($T_w = 400$ K) | + no-slip ($U = 0$) | } |
| | Top wall : Dirichlet Boundary Condition ($T_w = 400$ K) | + no-slip ($U = 0$) | |
| | Inlet : Dirichlet Boundary Condition ($T_{in} = 300$ K) | + $u(y) = \frac{3}{2} U_0 \left(1 - \left(\frac{y-l}{l} \right)^2 \right)$ | |
| | Outlet : No Boundary condition ($\partial_E = 0$) | | |
- fully-developed assumption (input)

$\rightarrow$ We can easily determine the boundary conditions at the bottom wall, top wall, and inlet from the problem conditions. At the outflow, where fluid leaves the domain, we do not need to impose any temperature or flux condition. Instead, following the assumption of high Peclet number and one-way behavior by parabolic PDE, I set $\partial_E = 0$. Since the flow is fully developed and Peclet number is assumed to be large, the influence from the outflow boundary is negligible.

(b) Discretized Equation

Since the given flow field must satisfy the continuity equation, $\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x_j} (\rho u_j) = 0$

the general differential equation : $\frac{\partial}{\partial t} (\rho \phi) + \frac{\partial}{\partial x_j} (\rho u_j \phi) = \frac{\partial}{\partial x_j} \left(\Gamma \frac{\partial \phi}{\partial x_j} \right) + \dot{S}$ (ϕ : Temperature)
 $\therefore$ steady $\therefore$ no source

$$\rightarrow \frac{\partial J_x}{\partial x} + \frac{\partial J_y}{\partial y} = 0 \quad (J_x \equiv \rho u \phi - \Gamma \frac{\partial \phi}{\partial x}, J_y \equiv \rho v \phi - \Gamma \frac{\partial \phi}{\partial y})$$

J_x, J_y : total convection plus diffusion fluxes

u, v : Velocity components in the x and y directions

+) ϕ_p is assumed to prevail over the whole control volume.

$$\int_s^n \int_w^e \frac{\partial J_x}{\partial x} dx dy + \int_s^n \int_w^e \frac{\partial J_y}{\partial y} dx dy = 0$$

$$(J_e - J_w) + (J_n - J_s) = 0 \quad \dots \textcircled{1}$$

$$\left(\begin{array}{l} J_e = \int_s^n J_x|_e dy, J_w = \int_s^n J_x|_w dy \\ J_n = \int_w^e J_y|_n dx, J_s = \int_w^e J_y|_s dx \end{array} \right)$$

$$\text{continuity equation : } \frac{\partial}{\partial x_j} (\rho u_j) = 0 \rightarrow \int_s^n \int_w^e \frac{\partial (\rho u)}{\partial x} dx dy + \int_s^n \int_w^e \frac{\partial (\rho v)}{\partial y} dx dy = 0$$

$$\{(\rho u)_e - (\rho u)_w\} \Delta y + \{(\rho v)_n - (\rho v)_s\} \Delta x = 0$$

$$(F_e - F_w) + (F_n - F_s) = 0 \quad \dots \textcircled{2}$$

$$(F_e = (\rho u)_e \Delta y, F_w = (\rho u)_w \Delta y, F_n = (\rho v)_n \Delta x, F_s = (\rho v)_s \Delta x)$$

$\therefore$ fully developed assumption

$$\rightarrow v = 0 \sim F_n = F_s = 0$$

$$u = u(y) = u_{max} \cdot \left(1 - \left(\frac{y-l}{l}\right)^2\right)$$

$$y \in [0, 2l], u_{max} = \frac{3}{2} u_0$$

$$= \frac{3}{2} \cdot \frac{\mu \cdot Re}{\rho \cdot (2l)}$$

$$\textcircled{1} - \textcircled{2} \times \phi_p \rightarrow [J_e - F_e \phi_p] - [J_w - F_w \phi_p] + [J_n - F_n \phi_p] - [J_s - F_s \phi_p]$$

$$J_e - F_e \phi_p = a_E (\phi_p - \phi_E), J_w - F_w \phi_p = a_W (\phi_W - \phi_p)$$

$$J_n - F_n \phi_p = a_N (\phi_p - \phi_N), J_s - F_s \phi_p = a_S (\phi_S - \phi_p)$$

$$a_E = D_e A(|P_E|) + [-F_e, 0]$$

$$a_W = D_w A(|P_W|) + [F_w, 0]$$

$$a_N = D_n A(|P_N|) + [-F_n, 0]$$

$$a_S = D_s A(|P_S|) + [F_s, 0]$$

$$D_e = \frac{\Gamma_e \Delta y}{(\delta x)_e}, D_w = \frac{\Gamma_w \Delta y}{(\delta x)_w}$$

$$D_n = \frac{\Gamma_n \Delta x}{(\delta x)_n}, D_s = \frac{\Gamma_s \Delta x}{(\delta x)_s}$$

$$P_e = \frac{F_e}{D_e}, P_w = \frac{F_w}{D_w}$$

$$P_n = \frac{F_n}{D_n}, P_s = \frac{F_s}{D_s}$$

Scheme	$A(P)$
Central difference	$1 - 0.5 P $
Upwind	1
Hybrid	$[[0, 1 - 0.5 P]]$
Power law	$[[0, (1 - 0.1 P)^4]]$

$$\rightarrow a_P \phi_p = a_E \phi_E + a_W \phi_W + a_N \phi_N + a_S \phi_S \quad (a_p = a_E + a_W + a_N + a_S)$$

: Discretized equation

** The 4 basic rules.

1. Rule #1. Consistency at control volume faces : Satisfied ($\therefore$ the thermal conductivity is uniform throughout the entire domain.)

2. Rule #2. Positive coefficients : Satisfied ($\therefore$ All coefficients are positive in the interior. $a_E, a_W, a_S, a_N > 0$)
 At the boundary, one of the coefficients may become zero, but since it is not included in the calculation, Rule #2 is always satisfied.

3. Rule #3. Negative - slope linearization of the source term : Satisfied ($\therefore$ no source exists.)

4. Rule #4. Sum of the neighbor coefficients : Satisfied ($\therefore a_p = a_E + a_W + a_N + a_S$)

* Scarborough criterion

I already showed that $a_p = \sum a_{nb}$ in the entire domain. (We don't have any source here.)

Thus, $\frac{\sum |a_{nb}|}{|a_p|} \leq 1$ is satisfied for all equations and < 1 for at least one equation (at the boundary).

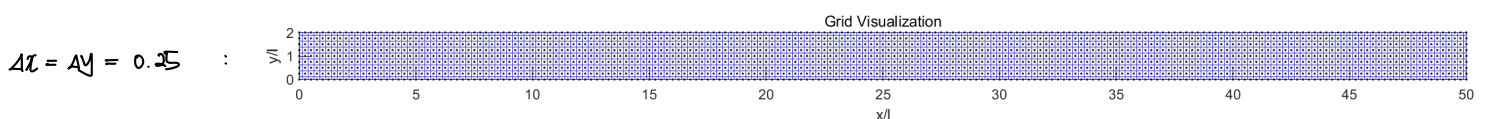
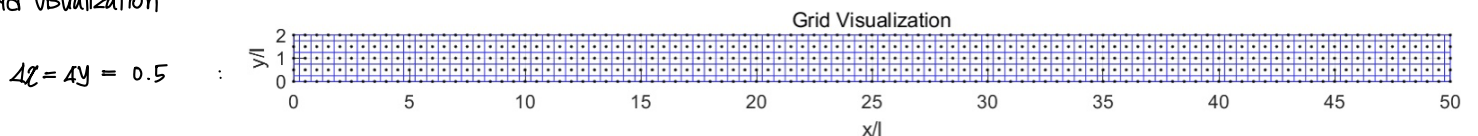
* Matlab coding

```

편집기 - Term_project_1_complete_20244109.m
grid_convergence_test_complete_20244109.m Term_project_1_complete_20244109.m +
1 % Matlab code for Term-Project #1 (c), (d)
2 % 20244109 기계공학과 권혁술
3 % grid test 용 코드는 grid_convergence_test_complete_20244109.m 에 존재
4 % 사용법
5 % 1) cell_size 선정 ex) 0.25 , 0.25/2 , 0.25/4 , 0.25/8
6 % 2) 대류 스킴 및 솔버 선택
7
8 clc; clear; close all;
9
10 %% Global Parameter
11 global rho cp l Uave Tin Tw
12
13 %% Material Property and Parameter
14 rho = 1.177; % kg/m^3
15 mu = 1.846e-5; % kg/m.s
16 cp = 1007; % J/kg.K
17 k = 0.02624; % W/m.K
18 Gamma = k/(rho*cp);
19
20 l = 1;
21 Lx = 50*1; Ly = 2*1;
22 Tw = 400; % wall temperature
23 Tin = 300; % inlet temperature
24 Re = 50;
25 Uave = Re*mu/(rho*l);
26 Umax = 1.5*Uave; % parabolic velocity profile
27
28 %% Convergence Criteria
29 tol_diff = 1e-6; % max|ΔT| tolerance
30 tol_resid = 1e-3; % normalized residual tolerance (0.1%)
31 maxit = 10000;
32
33 %% Under/Over-relaxation parameter
34 omega = 0.8; % for SOR
35 alpha = 1.0; % for Ext-Jacobi
36
37 %% Grid Generation
38 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
39 cell_size = 0.25; % Only change this value
40 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
41 half_c = cell_size/2; % size of the boundary half cell
42 x_cent = 0:cell_size:Lx; % location of the center points in x-dir
43 Nx = numel(x_cent); % number of the points in x-dir
44 y_cent = 0:cell_size:Ly; % location of the center points in y-dir
45 Ny = numel(y_cent); % number of the points in y-dir
46
47 dx_w = half_c*ones(1,Nx); % location of the west surfaces
48 dx_w(1)=0;
49 dx_e = half_c*ones(1,Nx); % location of the east surfaces
50 dx_e(end)=0;
51 dy_s = half_c*ones(Ny,1); % location of the south surfaces
52 dy_s(1)=0;
53 dy_n = half_c*ones(Ny,1); % location of the north surfaces
54 dy_n(end)=0;
55
56 x_faces = [x_cent - dx_w, x_cent(end)+dx_e(end)]; % location of the surfaces in x-dir
57 y_faces = [y_cent - dy_s, y_cent(end)+dy_n(end)]; % location of the surfaces in y-dir

```

* Grid visualization



```

58 %% Grid Visualization
59 figure;
60 hold on;
61 axis equal;
62 xlabel('x/l'); ylabel('y/l');
63 title('Grid Visualization');
64
65 % Plot cell boundaries
66 for i = 1:Nx
67     for j = 1:Ny
68         x_left = x_cent(i) - dx_w(i);
69         x_right = x_cent(i) + dx_e(i);
70         y_bottom = y_cent(j) - dy_s(j);
71         y_top = y_cent(j) + dy_n(j);
72
73         rectangle('Position', [x_left/l, y_bottom/l, (x_right - x_left)/l, (y_top - y_bottom)/l], ...
74             'EdgeColor', [0.2 0.2 0.8], 'LineWidth', 0.5);
75     end
76 end
77
78 % Optionally plot center points (dots)
79 [Xc, Yc] = meshgrid(x_cent, y_cent);
80 plot(Xc(:)/l, Yc(:)/l, 'k.', 'MarkerSize', 4);
81
82 xlim([x_cent(1), x_cent(end)]/l);
83 ylim([y_cent(1), y_cent(end)]/l);
84 grid on;
85

```

```

86 %% Inlet velocity
87
88 y_rel = (y_cent - 1)/l; % relative y-coordinate for the velocity u_profile
89 u_prof = Umax*(1 - y_rel.^2);
90

```

* fully-developed laminar flow

$$U_{ave} = \frac{1}{2l} \int_0^{2l} u(y) dy = \frac{2}{3} U_{max}$$

$$U_{max} = \frac{3}{2} U_{ave}$$

$$u(y) = U_{max} \left(1 - \left(\frac{y-l}{l} \right)^2 \right)$$

$$y \in [0, 2l]$$

```

91 %% Convection Schemes
92 Afuncs.Central = @(P) 1 - 0.5*abs(P);
93 Afuncs.Upwind = @(P) ones(size(P));
94 Afuncs.Hybrid = @(P) max(0, 1 - 0.5*abs(P));
95 Afuncs.PowerLaw = @(P) max(0, (1 - 0.1*abs(P))).^5;
96 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
97 schemeSel = 'PowerLaw'; % 선택: 'Central', 'Upwind', 'Hybrid', 'PowerLaw'
98 solverSel = 'SOR'; % 선택: 'Jacobi', 'GS', 'SOR', 'EJacobi', 'LineByLine', 'ADI'
99 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
100 Afunc = Afuncs.(schemeSel);
101
102 %% Coefficients for discretized equation
103 aE = zeros(Ny,Nx); aW = aE; aN = aE; aS = aE; aP = aE; b = zeros(Ny,Nx);
104 for j = 1:Ny
105     for i = 1:Nx
106         Ae = y_faces(j+1)-y_faces(j); Aw = Ae; % Ae = Aw = delta_y
107         An = x_faces(i+1)-x_faces(i); As = An; % An = As = delta_x
108
109         de = dx_e(i); % del_e : distance between node and east surface
110         dw = dx_w(i); % del_w : distance between node and west surface
111         dn = dy_n(j); % del_n : distance between node and north surface
112         ds = dy_s(j); % del_s : distance between node and south surface
113
114         Fe = rho*u_prof(j)*Ae; Fw = Fe;
115         Fn = 0; Fs = 0;
116
117         De = (de>0)*Gamma*Ae/de;
118         Dw = (dw>0)*Gamma*Aw/dw;
119         Dn = (dn>0)*Gamma*An/dn;
120         Ds = (ds>0)*Gamma*As/ds;
121
122         if i==Nx
123             ae = 0;
124         else
125             Pe = Fe/De; ae = De*Afunc(Pe) + max(-Fe,0);
126         end
127         if i==1
128             aw = 0;
129         else
130             Pw = Fw/Dw; aw = Dw*Afunc(Pw) + max(Fw,0);
131         end
132         if j==Ny
133             an = 0;
134         else
135             Pn = Fn/Dn; an = Dn*Afunc(Pn) + max(-Fn,0);
136         end
137         if j==1
138             as = 0;
139         else
140             Ps = Fs/Ds; as = Ds*Afunc(Ps) + max(Fs,0);
141         end
142
143         aE(j,i)=ae; aW(j,i)=aw;
144         aN(j,i)=an; aS(j,i)=as;
145         aP(j,i)=ae+aw+an+as + (Fe - Fw + Fn - Fs); % Fe-Fw+Fn-Fs = 0
146     end
147 end
148 coef = struct('aE',aE,'aW',aW,'aN',aN,'aS',aS,'aP',aP,'b',b);
149

```

%% Boundary Cndtions (Dirichlet)

T0 = Tin*ones(Ny,Nx);

T0(:,1) = Tin;

T0(1,:) = Tw;

T0(Ny,:) = Tw;

%% Solver selection

switch solverSel

case 'Jacobi'

T = jacobiSolve(coef, T0, tol_diff, tol_resid, maxit);

case 'GS'

T = gsSolve(coef, T0, tol_diff, tol_resid, maxit);

case 'SOR'

T = sorSolve(coef, T0, omega, tol_diff, tol_resid, maxit);

case 'EJacobi'

T = extJacobiSolve(coef, T0, alpha, tol_diff, tol_resid, maxit);

case 'LineByLine'

T = lineByLineSolve(coef, T0, tol_diff, tol_resid, maxit);

case 'ADI'

T = adiSolve(coef, T0, tol_diff, tol_resid, maxit);

otherwise

error('Unknown solver: %s', solverSel);

end

%% Problem(c) : Result Plotting

figure;

imagesc(x_cent/l, y_cent/l, T);

axis xy equal; colorbar;

xlabel('x/l'); ylabel('y/l');

title([schemeSel ' ' solverSel], 'Interpreter', 'none');

180

* Convergence Criteria

$$\left\{ \begin{array}{l} \text{normalized Residual (energy)} = \frac{\sum |a_p T_p - \sum a_{nb} T_{nb} - b|}{\sum \dot{m} c_p T_{in} + \sum \dot{m} c_p |T_w - T_m|} < 0.1\% \\ \text{maximum difference of } |T_{new} - T_{old}| < 1 \cdot 10^{-6} \end{array} \right.$$

At first, to determine convergence criteria I made that the solution had converged when the maximum change between the old and the updated temperature fields fell below a specified ϵ . I used $1 \cdot 10^{-6}$ for ϵ .

However, since heavy under-relaxation can create an illusion of convergence, I added a residual based criterion. I learned that 0.1% of {the sum of $\dot{m} c_p T_{in}$ + the sum of heat sources} can be a standard to choose ϵ .

So I adopted the normalized residual definition from the Fluent manual, and declare convergence when that residual drops below 0.1%.

```

214 %% Linear Solver(Jacobi, GS, SOR, EJ, LineByLine, ADI) %%
215 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
216 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
217
218 function T = jacobiSolve(coef, T, tol_diff, tol_resid, maxit)
219 global rho cp Uave Tin Tw
220 [Ny,Nx] = size(T);
221 for it = 1:maxit
222     Tnew = T; maxd = 0;
223     for j = 2:Ny-1
224         for i = 2:Nx
225             west = coef.aW(j,i)*T(j,i-1);
226             east = coef.aE(j,i)*T(j,min(i+1,Nx));
227             south = coef.aS(j,i)*T(j-1,i);
228             north = coef.aN(j,i)*T(j+1,i);
229             Tj = (west+east+south+north+coef.b(j,i)) / coef.aP(j,i);
230             maxd = max(maxd, abs(Tj - T(j,i)));
231             Tnew(j,i) = Tj;
232         end
233     end
234     T = Tnew;
235
236     % Residual 계산
237     residual = zeros(Ny,Nx);
238     for j = 2:Ny-1
239         for i = 2:Nx
240             residual(j,i) = abs( ...
241                 coef.aP(j,i)*T(j,i) ...
242                 - coef.aE(j,i)*T(j,min(i+1,Nx)) ...
243                 - coef.aW(j,i)*T(j,i-1) ...
244                 - coef.aN(j,i)*T(j+1,i) ...
245                 - coef.aS(j,i)*T(j-1,i) ...
246                 - coef.b(j,i) );
247         end
248     end
249     res_sum = sum(residual(:));
250     m_dot = rho * (2*pi) * Uave;
251     escale = m_dot * cp * Tin + m_dot * cp * abs(Tw - Tin);
252     norm_res = res_sum / escale;
253
254     % iteration마다 상태 출력
255     fprintf('Jacobi iter %4d: Max difference = %.2e, Residual = %.2e\n', ...
256         it, maxd, norm_res);
257
258     if maxd < tol_diff && norm_res < tol_resid
259         fprintf('Jacobi converged at iter %d\n\n', it);
260         return;
261     end
262 end
263 warning('Jacobi maxit reached: Max difference=%.2e, Residual=%.2e', maxd, norm_res);
264 end
265

```

Jacobi

Gauss-Seidel

```

266 function T = gsSolve(coef, T, tol_diff, tol_resid, maxit)
267 global rho cp Uave Tin Tw
268 [Ny,Nx] = size(T);
269 for it = 1:maxit
270     maxd = 0;
271     for j = 2:Ny-1
272         for i = 2:Nx
273             west = coef.aW(j,i)*T(j,i-1);
274             east = coef.aE(j,i)*T(j,min(i+1,Nx));
275             south = coef.aS(j,i)*T(j-1,i);
276             north = coef.aN(j,i)*T(j+1,i);
277             Tj = (west+east+south+north+coef.b(j,i)) / coef.aP(j,i);
278             maxd = max(maxd, abs(Tj - T(j,i)));
279             T(j,i) = Tj;
280         end
281     end
282
283     % Residual 계산
284     residual = zeros(Ny,Nx);
285     for j = 2:Ny-1
286         for i = 2:Nx
287             residual(j,i) = abs( ...
288                 coef.aP(j,i)*T(j,i) ...
289                 - coef.aE(j,i)*T(j,min(i+1,Nx)) ...
290                 - coef.aW(j,i)*T(j,i-1) ...
291                 - coef.aN(j,i)*T(j+1,i) ...
292                 - coef.aS(j,i)*T(j-1,i) ...
293                 - coef.b(j,i) );
294         end
295     end
296     res_sum = sum(residual(:));
297     m_dot = rho * (2*pi) * Uave;
298     escale = m_dot * cp * Tin + m_dot * cp * abs(Tw - Tin);
299     norm_res = res_sum / escale;
300
301     fprintf('GS iter %4d: Max difference = %.2e, Residual = %.2e\n', ...
302         it, maxd, norm_res);
303
304     if maxd < tol_diff && norm_res < tol_resid
305         fprintf('GS converged at iter %d\n\n', it);
306         return;
307     end
308 end
309 warning('GS maxit reached: Max difference=%.2e, Residual=%.2e', maxd, norm_res);
310 end

```

```

312 function T = sorSolve(coef, T, omega, tol_diff, tol_resid, maxit)
313     global rho cp l Uave Tin Tw
314     [Ny,Nx] = size(T);
315     for it = 1:maxit
316         maxd = 0;
317         for j = 2:Ny-1
318             for i = 2:Nx
319                 west = coef.aW(j,i)*T(j,i-1);
320                 east = coef.aE(j,i)*T(j,min(i+1,Nx));
321                 south = coef.aS(j,i)*T(j-1,i);
322                 north = coef.aN(j,i)*T(j+1,i);
323                 Tnew = (west+east+south+north+coef.b(j,i)) / coef.aP(j,i);
324                 Tj = T(j,i) + omega*(Tnew - T(j,i));
325                 maxd = max(maxd, abs(Tj - T(j,i)));
326                 T(j,i) = Tj;
327             end
328         end
329
330         % Residual 계산
331         residual = zeros(Ny,Nx);
332         for j = 2:Ny-1
333             for i = 2:Nx
334                 residual(j,i) = abs( ...
335                     coef.aP(j,i)*T(j,i) ...
336                     - coef.aE(j,i)*T(j,min(i+1,Nx)) ...
337                     - coef.aW(j,i)*T(j,i-1) ...
338                     - coef.aN(j,i)*T(j+1,i) ...
339                     - coef.aS(j,i)*T(j-1,i) ...
340                     - coef.b(j,i) );
341             end
342         end
343         res_sum = sum(residual(:));
344         m_dot = rho * (2*1) * Uave;
345         escale = m_dot * cp * Tin + m_dot * cp * abs(Tw - Tin);
346         norm_res = res_sum / escale;
347
348         fprintf('SOR iter %4d: Max difference = %.2e, Residual = %.2e\n', ...
349             it, maxd, norm_res);
350
351         if maxd < tol_diff && norm_res < tol_resid
352             fprintf('SOR converged at iter %d\n\n', it);
353             return;
354         end
355     end
356     warning('SOR maxit reached: Max difference=%.2e, Residual=%.2e', maxd, norm_res);
357 end

```

SOR

EJ

```

359 function T = extJacobiSolve(coef, T, alpha, tol_diff, tol_resid, maxit)
360     global rho cp l Uave Tin Tw
361     [Ny,Nx] = size(T);
362
363     for it = 1:maxit
364         Told = T;
365         maxd = 0;
366
367         % (1) Red 셀 업데이트 (i+j 짝수)
368         for j = 2:Ny-1
369             for i = 2:Nx
370                 if mod(i+j,2) == 0
371                     west = coef.aW(j,i)*Told(j,i-1);
372                     east = coef.aE(j,i)*Told(j,min(i+1,Nx));
373                     south = coef.aS(j,i)*Told(j-1,i);
374                     north = coef.aN(j,i)*Told(j+1,i);
375                     Tj = (west + east + south + north + coef.b(j,i)) / coef.aP(j,i);
376                     T(j,i) = Told(j,i) + alpha * (Tj - Told(j,i));
377                     maxd = max(maxd, abs(T(j,i) - Told(j,i)));
378                 end
379             end
380         end
381
382         % (2) Black 셀 업데이트 (i+j 홀수)
383         for j = 2:Ny-1
384             for i = 2:Nx
385                 if mod(i+j,2) == 1
386                     west = coef.aW(j,i)*Told(j,i-1);
387                     east = coef.aE(j,i)*Told(j,min(i+1,Nx));
388                     south = coef.aS(j,i)*Told(j-1,i);
389                     north = coef.aN(j,i)*Told(j+1,i);
390                     Tj = (west + east + south + north + coef.b(j,i)) / coef.aP(j,i);
391                     T(j,i) = Told(j,i) + alpha * (Tj - Told(j,i));
392                     maxd = max(maxd, abs(T(j,i) - Told(j,i)));
393                 end
394             end
395         end
396
397         % Residual 계산
398         residual = zeros(Ny,Nx);
399         for j = 2:Ny-1
400             for i = 2:Nx
401                 residual(j,i) = abs( ...
402                     coef.aP(j,i)*T(j,i) ...
403                     - coef.aE(j,i)*T(j,min(i+1,Nx)) ...
404                     - coef.aW(j,i)*T(j,i-1) ...
405                     - coef.aN(j,i)*T(j+1,i) ...
406                     - coef.aS(j,i)*T(j-1,i) ...
407                     - coef.b(j,i) );
408             end
409         end
410         res_sum = sum(residual(:));
411         m_dot = rho * (2*1) * Uave;
412         escale = m_dot * cp * Tin + m_dot * cp * abs(Tw - Tin);
413         norm_res = res_sum / escale;
414
415         fprintf('EJacobi iter %4d: Max difference = %.2e, Residual = %.2e\n', ...
416             it, maxd, norm_res);
417
418         % 수렴 체크 부분
419         if maxd < tol_diff && norm_res < tol_resid
420             fprintf('EJacobi converged at iter %d\n\n', it);
421             return;
422         end
423     end
424
425     warning('EJacobi maxit reached: Max difference=%.2e, Residual=%.2e', maxd, norm_res);
426 end
427

```

Line by Line

```

430 function T = lineByLineSolve(coef, T, tol_diff, tol_resid, maxit)
431     global rho cp l Uave Tin Tw
432     [Ny,Nx] = size(T);
433     for it = 1:maxit
434         maxd = 0;
435         for i = 2:Nx
436             Told = T(:,i);
437
438             % tri-diagonal Coefficients
439             a = zeros(Ny,1); b = zeros(Ny,1);
440             c = zeros(Ny,1); d = zeros(Ny,1);
441             a(1)=0; b(1)=1; c(1)=0; d(1)=Tw;
442             a(Ny)=0; b(Ny)=1; c(Ny)=0; d(Ny)=Tw;
443             for j = 2:Ny-1
444                 a(j) = coef.aS(j,i);
445                 b(j) = coef.aP(j,i);
446                 c(j) = coef.aN(j,i);
447                 Twest = T(j,i-1);
448                 Teast = T(j,min(i+1,Nx));
449                 d(j) = coef.b(j,i) + coef.aN(j,i)*Twest + coef.aE(j,i)*Teast;
450             end
451
452             % Thomas solver
453             Tcol = thomas(-a, b, -c, d);
454             T(:,i) = Tcol;
455             maxd = max(maxd, max(abs(Tcol - Told)));
456         end
457
458         % BC 재적용
459         T(:,1)=Tin; T(1,:)=Tw; T(Ny,:)=Tw;
460
461         % Residual 계산
462         residual = zeros(Ny,Nx);
463         for j = 2:Ny-1
464             for i = 2:Nx
465                 residual(j,i) = abs( ...
466                     coef.aP(j,i)*T(j,i) ...
467                     - coef.aE(j,i)*T(j,min(i+1,Nx)) ...
468                     - coef.aW(j,i)*T(j,i-1) ...
469                     - coef.aN(j,i)*T(j+1,i) ...
470                     - coef.aS(j,i)*T(j-1,i) ...
471                     - coef.b(j,i) );
472             end
473         end
474         res_sum = sum(residual(:));
475         m_dot = rho * (2*1) * Uave;
476         escale = m_dot * cp * Tin + m_dot * cp * abs(Tw - Tin);
477         norm_res = res_sum / escale;
478
479         fprintf('LBL iter %4d: Max difference = %.2e, Residual = %.2e\n', ...
480             it, maxd, norm_res);
481
482         if maxd < tol_diff && norm_res < tol_resid
483             fprintf('LBL converged at iter %d\n\n', it);
484             return;
485         end
486     end
487     warning('LBL maxit reached: Max difference=%.2e, Residual=%.2e', maxd, norm_res);
488 end

```

TDMA

```

560 function x = thomas(a,b,c,d)
561     n = length(d);
562     cpv = zeros(n,1); dp = zeros(n,1);
563     cpv(1)=c(1)/b(1); dp(1)=d(1)/b(1);
564     for k=2:n
565         denom = b(k) - a(k)*cpv(k-1);
566         cpv(k) = c(k)/denom;
567         dp(k) = (d(k) - a(k)*dp(k-1))/denom;
568     end
569     x = zeros(n,1); x(n)=dp(n);
570     for k=n-1:-1:1
571         x(k) = dp(k) - cpv(k)*x(k+1);
572     end
573 end

```

ADI

```
function T = adiSolve(coef, T, tol_diff, tol_resid, maxit)
global rho cp l Uave Tin Tw
[Ny,Nx] = size(T);
for it = 1:maxit
    T_old = T;

    % 1) y-implicit sweep
    for i = 2:Nx
        a=zeros(Ny,1); b=zeros(Ny,1); c=zeros(Ny,1); d=zeros(Ny,1);
        a(1)=0; b(1)=1; c(1)=0; d(1)=Tw;
        a(Ny)=0; b(Ny)=1; c(Ny)=0; d(Ny)=Tw;
        for j=2:Ny-1
            a(j)=coef.aS(j,i);
            b(j)=coef.aP(j,i);
            c(j)=coef.aN(j,i);
            d(j)=coef.b(j,i)+coef.aW(j,i)*T(j,i-1)+coef.aE(j,i)*T(j,min(i+1,Nx));
        end
        T(:,i)=thomas(-a,b,-c,d);
    end

    % BC 재적용
    T(:,1)=Tin; T(1,:)=Tw; T(Ny,:)=Tw;

    % 2) x-implicit sweep
    for j=2:Ny-1
        a=zeros(Nx,1); b=zeros(Nx,1); c=zeros(Nx,1); d=zeros(Nx,1);
        a(1)=0; b(1)=1; c(1)=0; d(1)=Tin;
        a(Nx)=0; b(Nx)=1; c(Nx)=0; d(Nx)=T(j,Nx);
        for i=2:Nx
            a(i)=coef.aW(j,i); b(i)=coef.aP(j,i); c(i)=coef.aE(j,i);
            d(i)=coef.b(j,i)+coef.aS(j,i)*T(j-1,i)+coef.aN(j,i)*T(j+1,i);
        end
        T(j,:)=thomas(-a,b,-c,d)';
    end

    % BC 재적용
    T(:,1)=Tin; T(1,:)=Tw; T(Ny,:)=Tw;

    % maxd 계산
    maxd = max(abs(T(:)-T_old(:)));

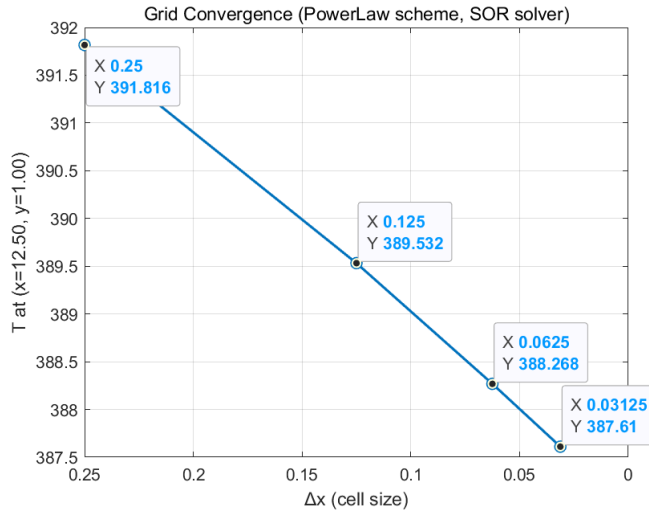
    % Residual 계산
    residual = zeros(Ny,Nx);
    for j=2:Ny-1
        for i=2:Nx
            residual(j,i) = abs( ...
                coef.aP(j,i)*T(j,i) ...
                - coef.aE(j,i)*T(j,min(i+1,Nx)) ...
                - coef.aW(j,i)*T(j,i-1) ...
                - coef.aN(j,i)*T(j+1,i) ...
                - coef.aS(j,i)*T(j-1,i) ...
                - coef.b(j,i) );
        end
    end
    res_sum = sum(residual(:));
    m_dot = rho * (2*l) * Uave;
    escale = m_dot * cp * Tin + m_dot * cp * abs(Tw - Tin);
    norm_res= res_sum / escale;

    fprintf('ADI iter %4d: Max difference = %.2e, Residual = %.2e\n', ...
        it, maxd, norm_res);

    if maxd < tol_diff && norm_res < tol_resid
        fprintf('ADI converged at iter %d\n\n', it);
        return;
    end
end
warning('ADI maxit reached: Max difference=%.2e, Residual=%.2e', maxd, norm_res);
end
```

(c) Temperature distribution when $Re = 50$

1) Grid convergence test



Division	dx	T_monitor	RelErr_to_next(%)	RelErr_to_fine(%)
1	0.2500	391.816	0.59	1.09
2	0.1250	389.532	0.33	0.50
4	0.0625	388.268	0.17	0.17
8	0.0312	387.610	N/A	0.00

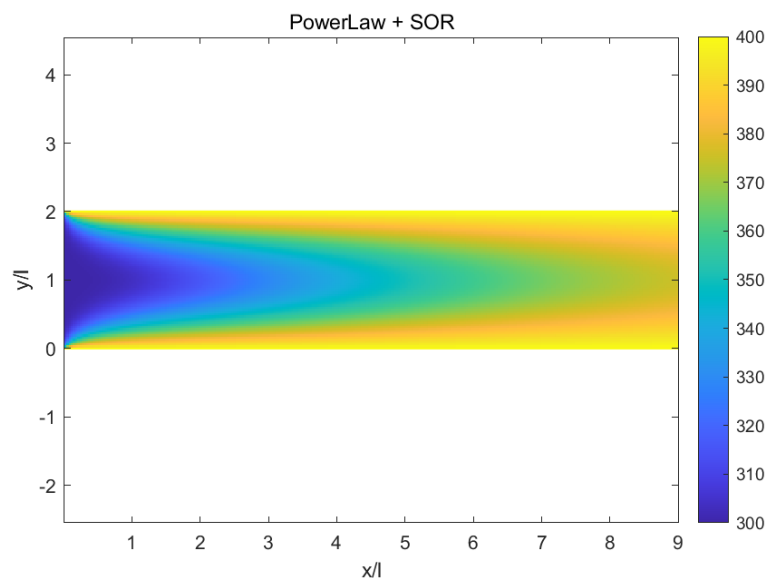
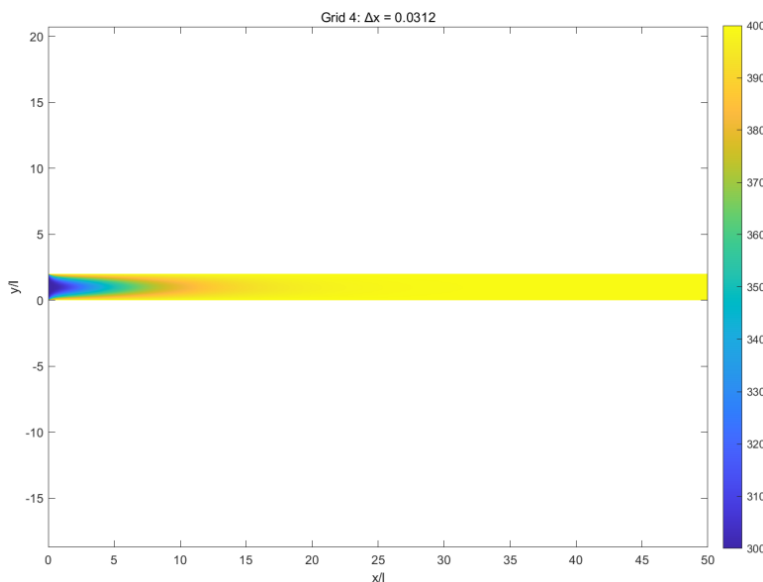
The initial grid size was set to 0.25 and then successively halved to perform the test with four different resolutions: 0.25, 0.125, 0.0625, and 0.03125. For the grid convergence test, the location of interest was chosen at $x = 50l \times 0.25 = 12.5$ ($l = 1$) along the x-axis and $y = 2l \times 0.5 = 1$ ($l = 1$).

For each grid size, the temperature was extracted at the cell closest to this position and used for comparison.

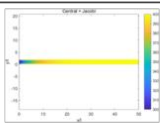
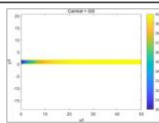
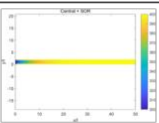
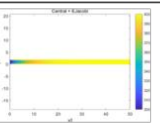
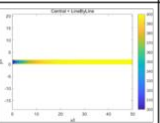
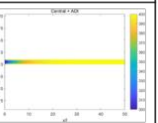
Based on the grid convergence results, using a cell size of around 0.0625 (which is 0.25 divided by 4) already gives a sufficiently accurate result, with the relative error to the finest grid falling below 0.2%. (The solution obtained with a cell size of 0.03125 was considered as the reference solution.)

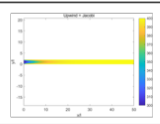
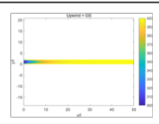
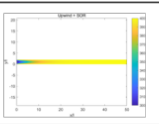
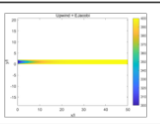
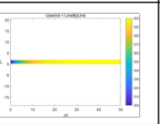
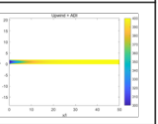
This suggests that further refinement may not be necessary for practical purposes. The following shows the temperature distribution calculated using a grid size of 0.03125. (Power Law + SOR, under-relaxation: 0.8, $Re = 50$) The temperature distribution exhibits a steep gradient near the inlet and along the walls, which gradually diminishes downstream as the thermal boundary layer develops.

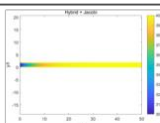
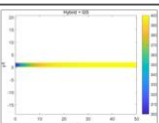
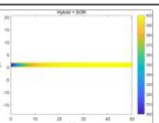
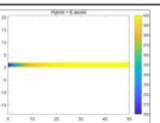
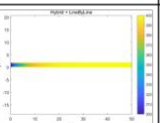
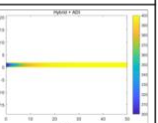
The thermal boundary layer grows along the channel walls and gradually thickens downstream until the temperature field becomes fully developed around $x/l \approx 4$.

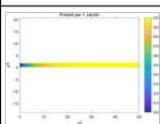
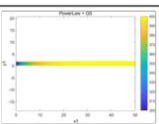
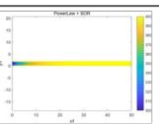
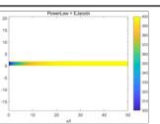
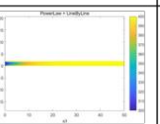
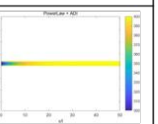


2) A comparison of different convection schemes and solvers used in the simulation ($Re = 50$)

	Jacobi	GS	SOR	EJ	LBL	ADI
Central Difference Scheme						
Iteration count	4118	1708	296 ($w=1.5$)	4118 ($a=1.0$)	677	423

	Jacobi	GS	SOR	EJ	LBL	ADI
Upwind Scheme						
Iteration count	5000	2162	224 ($w=1.6$)	5000 ($a=1.0$)	1128	567

	Jacobi	GS	SOR	EJ	LBL	ADI
Hybrid Scheme						
Iteration count	4118	1708	296 ($w=1.5$)	4118 ($a=1.0$)	677	423

	Jacobi	GS	SOR	EJ	LBL	ADI
Power Law Scheme						
Iteration count	4257	1779	321 ($w=1.5$)	4257 ($a=1.0$)	747	450

Based on the grid convergence study, a cell size of $0.25/4$ (i.e., 0.0625) was used for all simulations in this section.

In terms of convergence speed, the SOR, LBL, and ADI solvers consistently showed much faster performance than the basic Jacobi or GS solvers. For instance, using the Power Law scheme, the SOR solver required only 321 iterations, while Jacobi took 4257 iterations to reach convergence.

The Upwind scheme resulted in the slowest convergence when using basic solvers like Jacobi or EJ, both reaching the iteration limit (5000). In contrast, the SOR solver converged in just 224 iterations with an overrelaxation factor of 1.6.

Overall, the ADI and LBL solvers demonstrated a good balance between speed and stability across all convection schemes, while the SOR solver achieved the best performance when an appropriate relaxation factor was used.

(d) Comparison of the results for the Nusselt Number development with available data

To investigate the development of the Nusselt number between parallel plates, we referred to the paper by *Nickolay and Martin (2002)*, which provides a detailed background on heat transfer in hydrodynamically developed laminar flows. While the authors present an improved approximation for the Nusselt number, it requires solving eigenvalue problems, making it less practical for this study. Therefore, instead of using their formulation, we adopted the piecewise-defined correlation originally proposed by *Shah and London (1978)*, which is also cited in their paper.

Shah and London [3] propose the piecewise defined function

$$Nu_{SL} = \begin{cases} Nu_1 + 0.0235Gz & Gz < 166.\bar{6} \\ Nu_2 + 0.6 & 166.\bar{6} \geq Gz < 2000 \\ Nu_2 & Gz \geq 2000 \end{cases} \quad (3)$$

with

$$Nu_1 = \lim_{Gz \rightarrow 0} (Nu) = 7.541 \quad \text{and} \quad Nu_2 = \lim_{Gz \rightarrow \infty} (Nu) = 1.849Gz^{1/3} \quad (4)$$

for the mean Nusselt number and

$$Nu_{SL,x} = \begin{cases} Nu_1 + 6.874(Gz/1000)^{0.488} \exp\left(-\frac{245}{Gz}\right) & Gz < 1000 \\ 2/3Nu_2 + 0.4 & Gz \geq 1000 \end{cases} \quad (5)$$

for the local Nusselt number.

$$Re = \frac{\dot{M}d_h}{\mu A}, \quad Pr = \frac{c_p \mu}{\lambda} \quad \Rightarrow \quad Gz = \frac{Re \cdot Pr \cdot d_h}{x} = \frac{2\dot{M}d_h}{\lambda Bx}$$

Fluid & Thermal Properties

- μ : Dynamic viscosity [kg/m · s]
- c : Specific heat at constant pressure [J/(kg · K)]
- λ : Thermal conductivity [W/(m · K)]

Geometric Properties

- d_h : Hydraulic diameter [m], usually $d_h = 2s$
- B : Width of the plate [m]
- L : Channel (duct) length [m]
- s : Half gap between plates [m], or half channel height

Flow Parameters

- $\dot{M}$: Mass flow rate [kg/s]
- $u(y)$: Velocity profile [m/s]
- Re : Reynolds number $Re = \frac{\rho u_{avg} d_h}{\mu}$
- Pr : Prandtl number $Pr = \frac{c_p \mu}{\lambda}$

Dimensionless Groups

- Gz : Graetz number $= \frac{2\dot{M}d_h}{\lambda BL}$ or $= \frac{Re \cdot Pr \cdot d_h}{x}$
- Nu : Nusselt number $= \frac{\alpha d_h}{\lambda}$
- $Nu_{SL}, Nu_{SL,x}$: Shah & London empirical correlations for average and local Nusselt number

First, the bulk mean temperature was calculated to numerically determine the local Nusselt number. Regions with higher velocities have a greater influence on the mean temperature.

$$T_{bulk}(x_i) = \frac{\int_0^{2l} u(y) T(x_i, y) dy}{\int_0^{2l} u(y) dy} \quad h_x = -k \frac{\partial T}{\partial y} \Big|_{wall} \quad T_w - T_{bulk}$$

$$Nu_x = \frac{h_x D_h}{k}$$

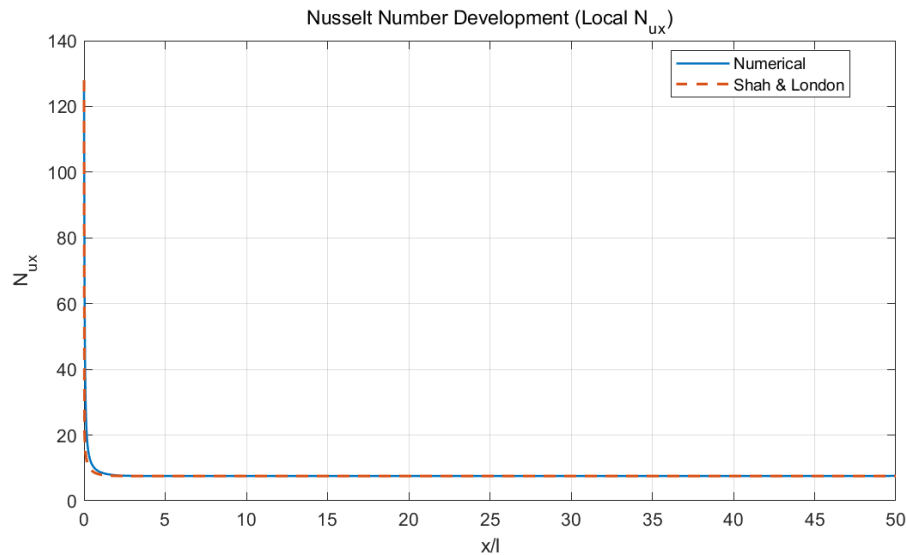
The local Nusselt number obtained from the numerical solution was calculated as follows. Then, the empirical correlation provided in the reference was directly applied. To avoid the singularity at $x=0$, the numerically computed value was used at the inlet instead of the empirical formula.

```

151 %% Problem(d) : Nusselt Number Development
152
153 Pr = cp * mu / k; % Prandtl Number
154 Dh = 4*1; % Hydraulic diameter (4*A/P = 4*(21*1)/(2*1))
155
156 % 1) Local Nusselt Number Calculation
157 Nu_loc = zeros(1, Nx);
158 T_mean = zeros(1, Nx);
159 for i = 1:Nx
160 % Bulk mean temperature calculation at each axial position
161 T_mean(i) = trapz(y_cent, u_prof .* T(:,i)) / trapz(y_cent, u_prof);
162
163 % Approximate temperature gradient at the wall using first cell center
164 dTdy_wall = (T(2,i) - Tw) / (y_cent(2) - y_cent(1));
165 h_x = -k * dTdy_wall / (Tw - T_mean(i));
166 Nu_loc(i) = h_x * Dh / k;
167 end
168
169 % 2) Numerically averaged Nusselt Number
170 Nu_avg_num = trapz(x_cent, Nu_loc) / Lx;
171
172 %% Comparison with available data (Numerical vs Shah & London correlation)
173 % 1) Graetz number and asymptotic Nusselt numbers
174 Gz = (Re * Pr * Dh) ./ x_cent;
175 Nu1 = 7.541;
176 Nu2 = 1.849 * Gz.^(1/3);
177
178 % 2) Shah & London local piecewise correlation
179 Nu_SL_x = zeros(1,Nx);
180 for i = 1:Nx
181 if x_cent(i)==0
182 Nu_SL_x(i) = Nu_loc(1); % Special handling at inlet (to avoid singularity)
183 elseif Gz(i) < 1000
184 Nu_SL_x(i) = Nu1 + 6.874*(Gz(i)/1000)^0.488 * exp(-245/Gz(i));
185 else
186 Nu_SL_x(i) = 2/3*Nu2(i) + 0.4;
187 end
188 end

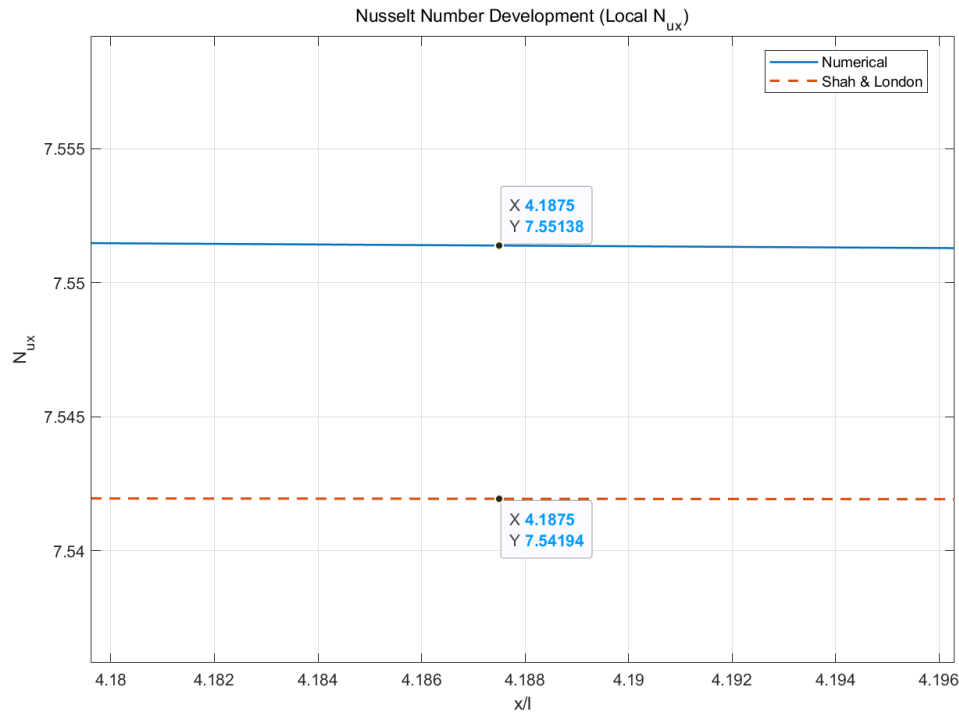
```

The numerical Nusselt number was computed using a grid size of 0.03125 with the Power Law scheme and SOR solver (under-relaxation : 0.8, $Re = 50$)



As the flow develops downstream, the local Nusselt number gradually decreases and approaches the asymptotic value of approximately **7.54**, which indicates that the thermal boundary layer has become fully developed. This confirms good agreement between the numerical solution and the analytical prediction.

Based on the variation in the local Nusselt number, the thermally fully-developed condition is observed to start at approximately $x/l \approx 4.19$. This location was determined by identifying the point where the relative change in Nu_x falls below 0.01% over consecutive axial positions, indicating that the local heat transfer coefficient has stabilized.



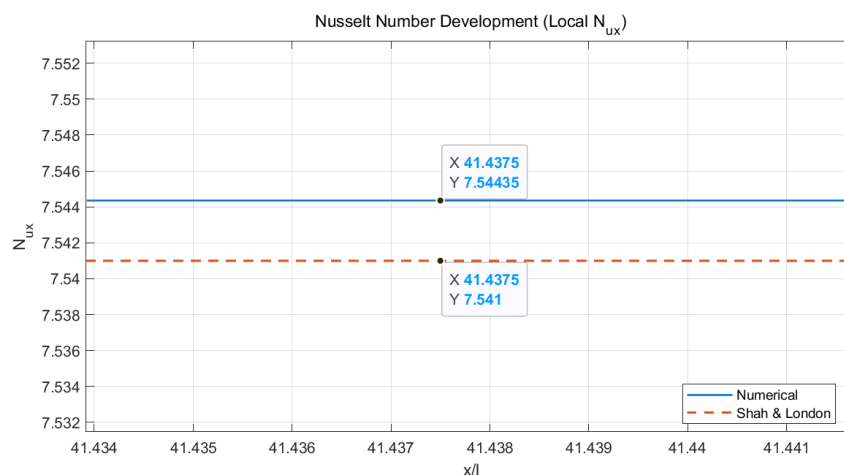
The theoretical prediction for the thermal entrance length, calculated using $L_{h,t} \approx 0.05 \cdot Re \cdot Pr \cdot Dh$, yields a value of $L_{h,t} / l = 3.5$ for $Re = 50$, $Pr \approx 0.71$, and $Dh = 2l$.

While the simulation result is slightly higher, the discrepancy can be attributed to several factors:

- 1) **Numerical diffusion**, especially from first-order schemes like upwind, may smooth out gradients and delay the apparent stabilization of the Nusselt number.
- 2) **Mesh resolution** in the streamwise direction may not be fine enough to capture the transition sharply, particularly near the thermal boundary layer edge.
- 3) **Threshold sensitivity**: The definition of "fully developed" based on a 1% change is arbitrary and may vary depending on the strictness of the criterion.

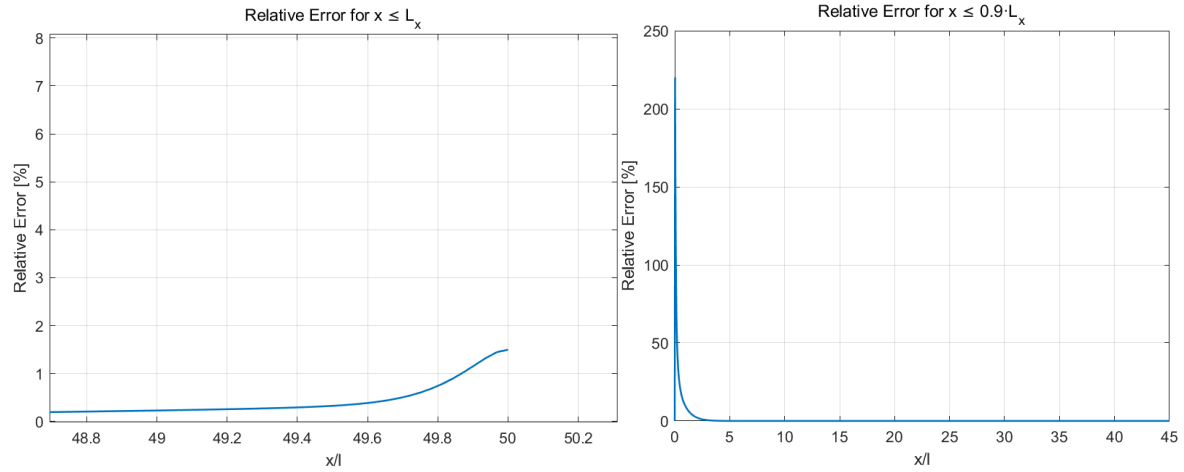
Despite the small difference, the simulation result at $x/l \approx 4.19$ is in close agreement with theory and confirms that the numerical model accurately captures the development of the thermal boundary layer.

According to the analytical correlation, the Nusselt number asymptotically approaches 7.541, while the numerical result converges to approximately 7.544. This slight discrepancy is likely due to insufficient grid refinement near the wall, which may have introduced errors in the temperature gradient calculation.



The relative error was evaluated up to the location corresponding to 90% of the channel length. The error was approximately 0.094% in this region. Near the outlet, where the temperature variation is minimal, the error increases to around 1%, so the comparison was limited to $x/l \leq 0.9$.

The relative error was defined as $\epsilon = \frac{Nu_{num} - Nu_{ref}}{Nu_{ref}} \times 100(\%)$.



Overall, the numerical results show excellent agreement with the analytical correlation. The remaining discrepancy, mainly attributed to grid resolution near the walls, can be further reduced through mesh refinement. These findings confirm both the accuracy and robustness of the numerical method in predicting local heat transfer behavior in thermally developing laminar flow between parallel plates.